1. Project Overview

Project Title

LLM-driven Structural Modeling and Analysis Pipeline using Open-Source FEM Tools (Code_Aster, Gmsh)

Partnership Context

This project is part of a collaborative applied research and training initiative between S.T.A. DATA S.r.l., a structural engineering software company based in Turin, and Politecnico di Torino. The activity takes place within a 250-hour advanced training program aimed at exploring innovative digital tools for the automation of structural modeling and analysis. The collaboration serves both educational and experimental purposes, combining industrial know-how with academic research capability.

Team:

- **Amir Yarmohamadi**
- **Saeideh Mohammadikish**

Both participants hold MSc degrees in Earthquake Engineering, with backgrounds in numerical modeling and seismic performance assessment, and are currently pursuing a second MSc at Politecnico di Torino focused on Digital Skills(Artificial Intelligence, Programming, and Data Science). This interdisciplinary expertise has allowed the team to act as an effective bridge between traditional structural analysis workflows and AI-based automation.

Motivation and Industrial Context

The company's original training proposal was conceptual, to explore whether Large Language Models (LLMs) could assist users in communicating with engineering software in natural language. No predefined workflow, data, or software integration architecture was provided; the starting point was a blank slate focused on feasibility and learning. The expectation was limited to a conceptual feasibility discussion or a manual demonstration using open-source solvers (Code_Aster, Gmsh).

Project Expansion and Added Value

Throughout the development, the project surpassed the initial expectations. Instead of producing only theoretical notes or mockups, the team designed and implemented a working technical pipeline that connects:

1. **LLM interface** → for interpreting natural-language instructions;
2. **Gmsh** → for automatic geometry and mesh generation;
3. **Code_Aster** → for running real finite element analysis;

4. **Custom Python modules** (run_aster.py, Visualizer.py) → for solver execution and result visualization.

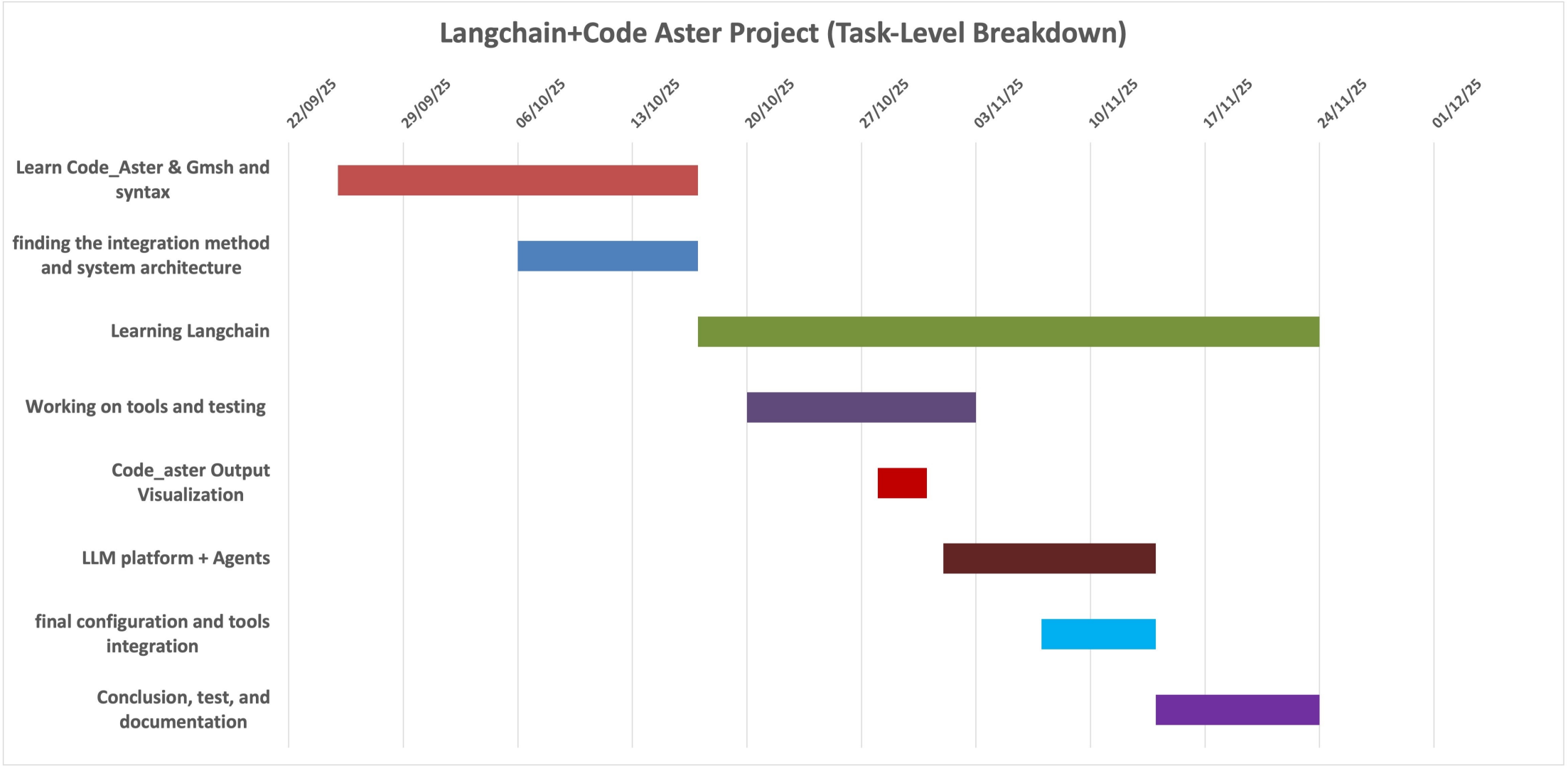
In essence, what was initially expected as a *feasibility training exercise* evolved into a functional prototype capable of automatically generating, executing, and visualizing a structural model.

Goal and Scope

The current phase focuses on a Proof of Concept (PoC), the automation of a simple beam model. The user provides geometric of a beam in natural language; the system builds the mesh, runs the solver, and visualizes displacements fields. This demonstration serves as the foundation for future extensions, including multi-element structures, 3D models, and integration with the company's proprietary software ecosystem.

2. Work Breakdown Structure (WBS) and Project Timeline

Week / Date Range (2025)	Phase	Key Focus	Status
Sep 25 – Oct 1	Phase 1 – Exploration	Familiarization with Code_Aster and Gmsh; understanding input/output files	Completed
Oct 6 – Oct 10	Phase 2 – Integration Method	Finding the integration method and system architecture	Completed
Oct 13 – Oct 17	Phase 3 – Core Reaserch	Learning Langchain	In Progress
Oct 20 – Oct 24	Phase 4 – Iniatial integration requaierment	The implementation of tools and testing for Langchain	Completed
Oct 27 – Oct 31	Phase 4 – Running Code_Aster	Code_aster Output Visualization	In Progress
Nov 3 – Nov 7	Phase 5 – Integration LLM	LLM platform + Agents	Planned
Nov 10 – Nov 14	Phase 6 – Agent Integration	Final configuration and tools integration	Planned
Nov 17 – Nov 21	Phase 6 – Documentation & Delivery	Conclusion, test, and documentation	Planned


Langchain+Code Aster Project (Task-Level Breakdown)
22/09/25
29/09/25
06/10/25
13/10/25
20/10/25
27/10/25
03/11/25
10/11/25
17/11/25
24/11/25
01/12/25
Learn Code_Aster & Gmsh and syntax
finding the integration method and system architecture
Learning Langchain
Working on tools and testing
Code_aster Output Visualization
LLM platform + Agents
final configuration and tools integration
Conclusion, test, and documentation

3. Deliverables & Validation

The expected outcomes of this 250-hour training and feasibility project are intentionally realistic and aligned with its educational scope. The goal is not to develop a commercial or production-ready system, but to build and demonstrate a working proof of concept that connects modern AI techniques with an open-source structural analysis tool.

The main deliverable is a functional prototype capable of showing the feasibility of an automated pipeline linking natural-language input to a structural solver through intermediate modules.
In practice, this means that a user can describe a simple structural problem (for example, a single beam under load), and the system, through the interaction between the LLM, Gmsh, and Code_Aster, can automatically:

1. generate the necessary geometry and mesh files,
2. create the solver input (.comm) file,
3. run the structural analysis,
4. and visualize the numerical output (e.g., nodal displacements).

In addition to this demonstrator, the project aims to deliver a set of reusable Python tools that form the backbone of this workflow. These include scripts for solver execution (run_aster.py) and result visualization (Visualizer.py), both of which are modular, transparent, and ready to be integrated into future developments.

A conceptual integration with LangChain is also part of the deliverable set: the system could be able to interpret simple user requests in natural language and convert them into structural model parameters. This integration does not need to be exhaustive or fully autonomous; it is sufficient to show that the approach is viable and that communication between the AI and the solver can occur without manual intervention.